



FIREBRAND

LYCEUM EDUCATION GROUP

Conditional Logic in Power Apps

Power Platform Pathway | Session 2

Session Recording & Professional Expectations

Session Recording

- All training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Conditional Logic and Decision Structures

Lesson Objectives

Apply

- Apply conditional logic within Power Apps formulas

Analyse

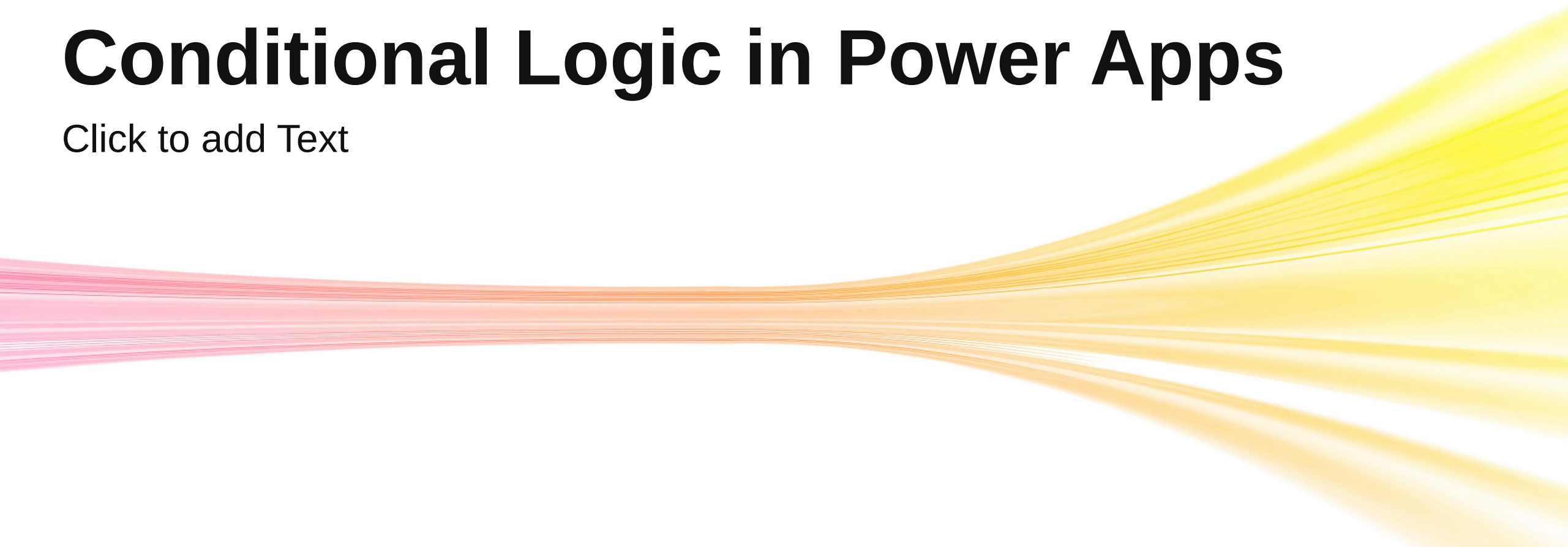
- Analyse decision logic using flowcharts

Evaluate

- Evaluate logic for clarity and usability

Conditional Logic in Power Apps

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that flow from the left side of the slide towards the right. The lines are colored in a gradient, starting with shades of pink and light orange on the left, transitioning through orange, and finally becoming bright yellow on the right. The lines have a soft, ethereal quality, resembling smoke or liquid motion.

The If() Function

- If() evaluates a condition and returns one value if true, another if false

Code Example:

```
// Label.Text property  
If(  
    TextInput1.Text = "",  
    "Please enter a value",  
    "Thank you!"  
)
```

Business Rules & Eligibility Logic

- Business logic often dictates access or rights based on profile attributes rather than raw numbers
- Understanding software maintenance means writing formulas that can adapt when corporate compliance or eligibility policies change

Code Example (User Role Eligibility):

// Button.DisplayMode property based on role eligibility

```
If(  
    User().Email in ApprovedAdminListCollection,  
    DisplayMode.Edit,  
    DisplayMode.View)
```

The If() Function: Condition, True, False

- **The Power Fx Model:** Power Fx does not use explicit if, else if, or else keywords. Instead, it uses a single, flattened If() function
- **Sequential Arguments:** The function takes sequential pairs of arguments: If(Condition1, Result1, Condition2, Result2, DefaultResult)
- **Default Catch-All:** The final argument with no paired condition acts as your traditional else statement

Code Example:

```
// Power Fx reads these pairs from left to right
If(
  Value(TextInput_Age.Text) < 13, "Child",
  Value(TextInput_Age.Text) < 18, "Teenager",
  "Adult"
)
```

Logical Operators in Power Fx

- **AND Logic (&& or And):** Evaluates to true only if both side parameters match
- **OR Logic (|| or Or):** Evaluates to true if at least one side condition evaluates to true
- **NOT Logic (! or Not):** Flips a boolean state from true to false, or vice-versa
- **Flexibility:** You can use traditional programming short-hand symbols (&&, ||, !) or text keywords (And, Or, Not) interchangeably

```
// Evaluates multiple states inline to set control visibility or execution
If(
    Value(TextInput_Age.Text) >= 18 && Toggle_HasTicket.Value,
    "Entry allowed",
    "Entry denied"
)
```

Real-World Rules to Declarative Formulas

The Business Rules:

An online e-commerce platform automatically calculates checkout discounts based on order basket totals:

- **Rule 1:** Orders under £50 receive a 0% discount
- **Rule 2:** Orders between £50 and £100 receive a 10% discount
- **Rule 3:** Premium orders over £100 receive a 20% discount

The Power Fx Interpretation:

- **Declarative Binding:** Instead of running an imperative function to calculate a value on click, we assign this logic directly to the target control's property (e.g., Label.Text)
- **Order of Evaluation:** The logic must target specific thresholds cleanly so broad nets don't accidentally swallow complex criteria

Pitfalls of Poor Conditional Ordering

The Danger of Overly Broad Nets

// WARNING: This formula breaks business rules due to poor ordering!

```
If(  
    Value(TextInput_OrderTotal.Text) >= 0, 0.00,    // Catches EVERYTHING first!  
    Value(TextInput_OrderTotal.Text) >= 50, 0.10,    // Unreachable expression  
    Value(TextInput_OrderTotal.Text) >= 100, 0.20    // Unreachable expression  
)
```

Clean & Maintainable Structure

Logically Ordered Power Fx Conditions

// CLEAN: Specific-to-general ordering maintains intended business logic

```
If(  
    Value(TextInput_OrderTotal.Text) >= 100, 0.20, // Narrowest, highest-priority target  
    Value(TextInput_OrderTotal.Text) >= 50, 0.10, // Mid-range fallback threshold  
    0.00 // General safety net / default state  
)
```

Design Principles for Code Maintainability



**SPECIFIC TO
GENERAL**



FAIL FAST



**READABILITY OVER
CLEVERNESS**

Designing Logic Before Coding

Map out decision paths as a flowchart or pseudocode before touching the Power Apps Maker Portal

This avoids missed cases, complex logic and overly overlapping validation gaps

Convert each decision point directly inside a flattened Power Fx If() expression

Right vs. Wrong (Conditional Logic)

Overly Complex / Wrong Order:

```
If(Value(Amount.Text)>1000, "High", If(
Value(Amount.Text)>500, If(Value(Amount.
t.Text)>100, "Medium", "Low"), "Low"))
```

Corrected, Logically Ordered:

```
If(
    Value(Amount.Text) > 1000,
    "High",
    Value(Amount.Text) > 100,
    "Medium",
    "Low"
)
```

Why is the second condition in the flawed version never reached?

The second condition is never reached because **conditional structures evaluate from top to bottom and stop at the first true match**

Visual Modelling: Mapping Real-World Logic

The Mission

- Translate abstract business rules into concrete, visual blueprints using **flowcharts** or **pseudocode** before writing any Power Fx formulas

Choose Your Scenario

Select **one** of the following real-world systems to model:

- **Login Validation:** Check if a username exists, verify the password, and handle account lockouts after 3 failed attempts
- **Discount Calculation:** Evaluate order totals to apply 0%, 10%, or 20% discounts accurately without overlapping ranges
- **Simple Eligibility Checker:** Determine if a user meets multi-variable criteria (e.g., age requirements combined with ticket ownership)

Your Tasks & Deliverables

Execution Steps

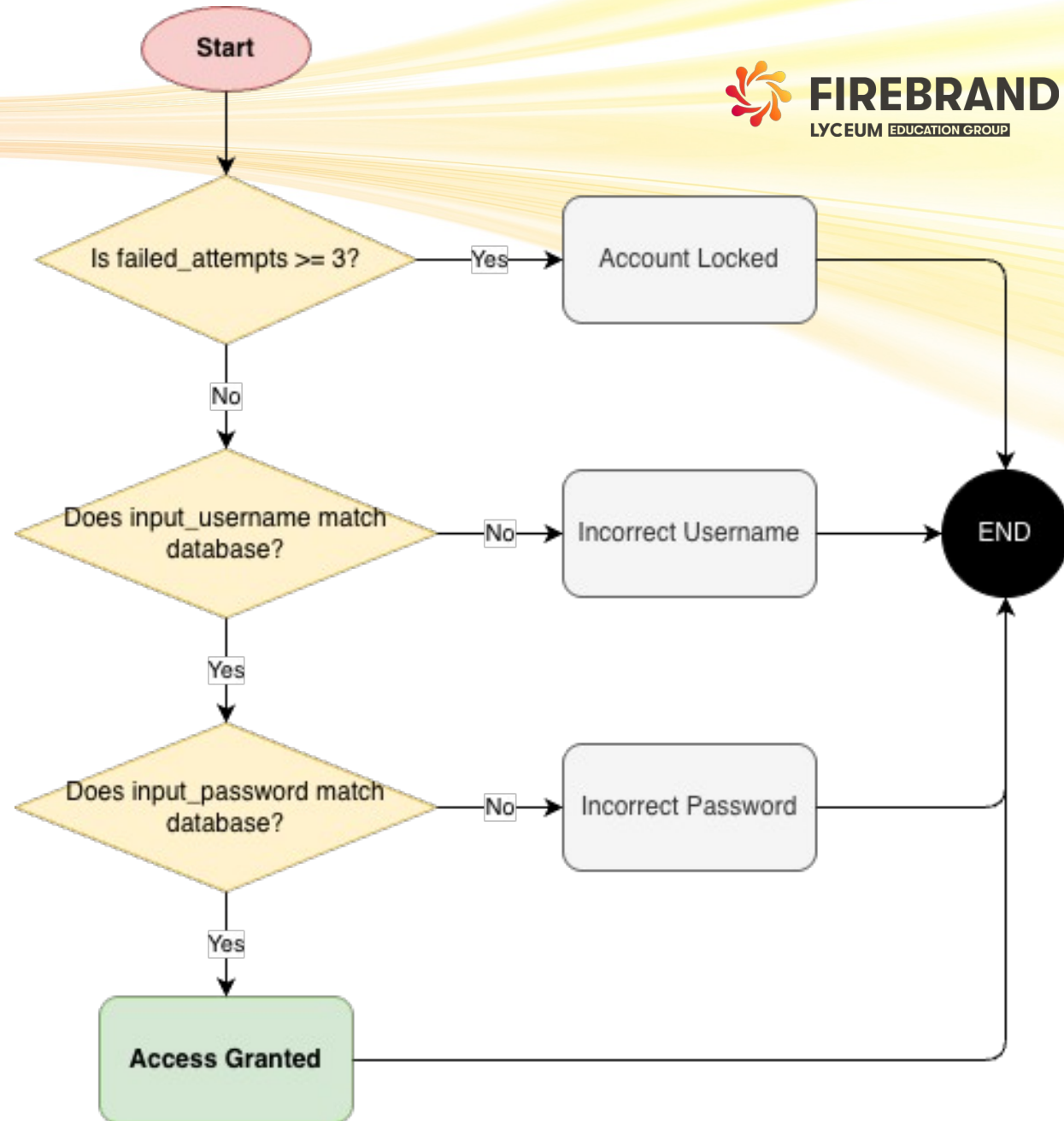
- **Draft the Model:** Sketch a visual flowchart (using diamonds for decisions) or write sequential pseudocode
- **Seal Every Leak:** Ensure all decision paths are complete, closed, and clearly defined (every "Yes" needs a matching "No")
- **Trace the Extremes:** Test your model with boundary values (e.g., exactly 0, exactly 18, or negative numbers) to catch hidden bugs

Expected Outcome

- A **clear, logical visual model** that fully represents the entire decision-making process
- A blueprint that a fellow programmer could easily read and convert into Power Fx syntax without guessing

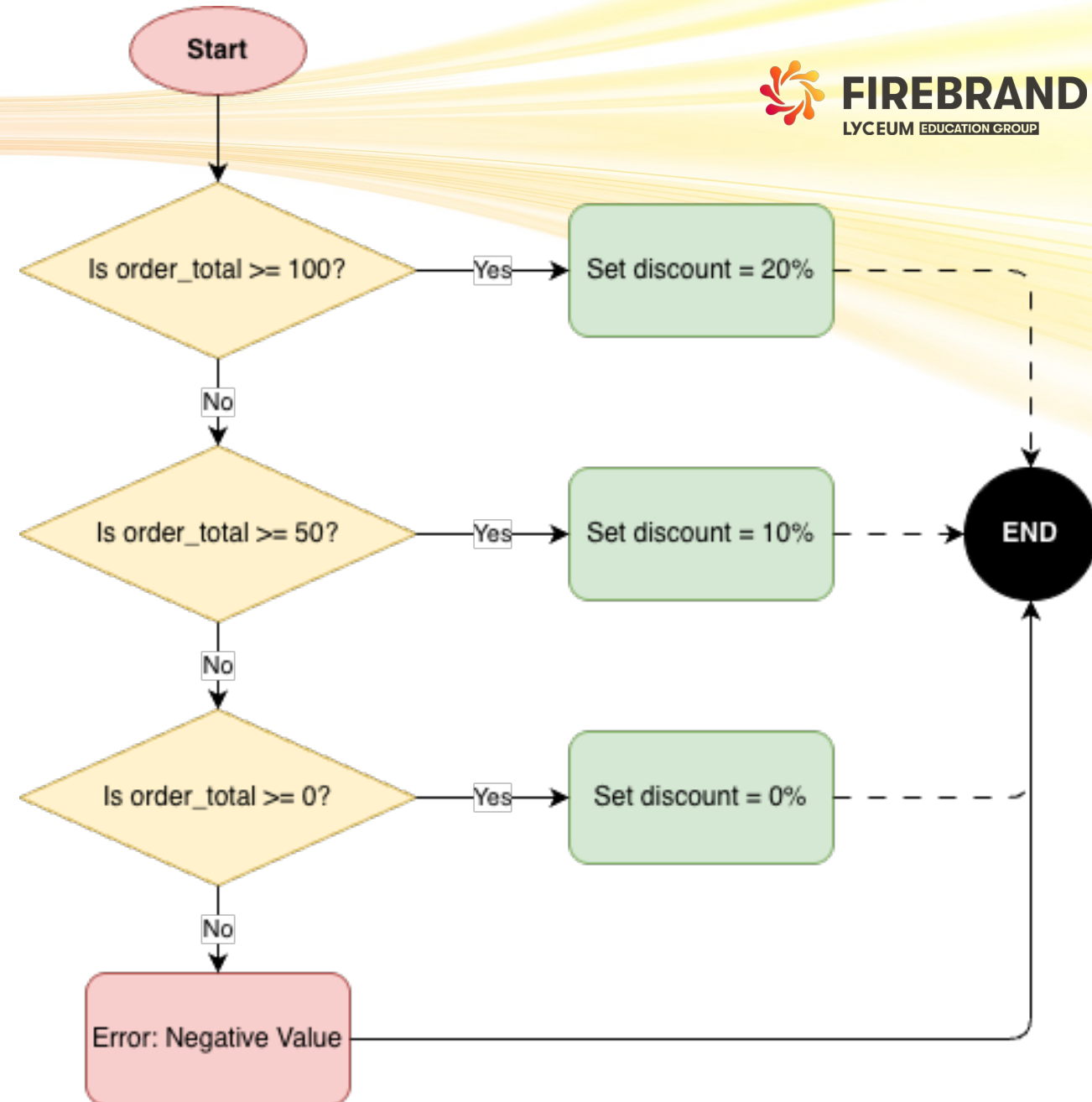
Login Validation

```
IF failed_attempts >= 3 THEN  
    DISPLAY "Account Locked"  
ELSE IF input_username != database_username THEN  
    DISPLAY "Incorrect Username"  
ELSE IF input_password != database_password THEN  
    DISPLAY "Incorrect Password"  
ELSE  
    DISPLAY "Access Granted"  
ENDIF
```



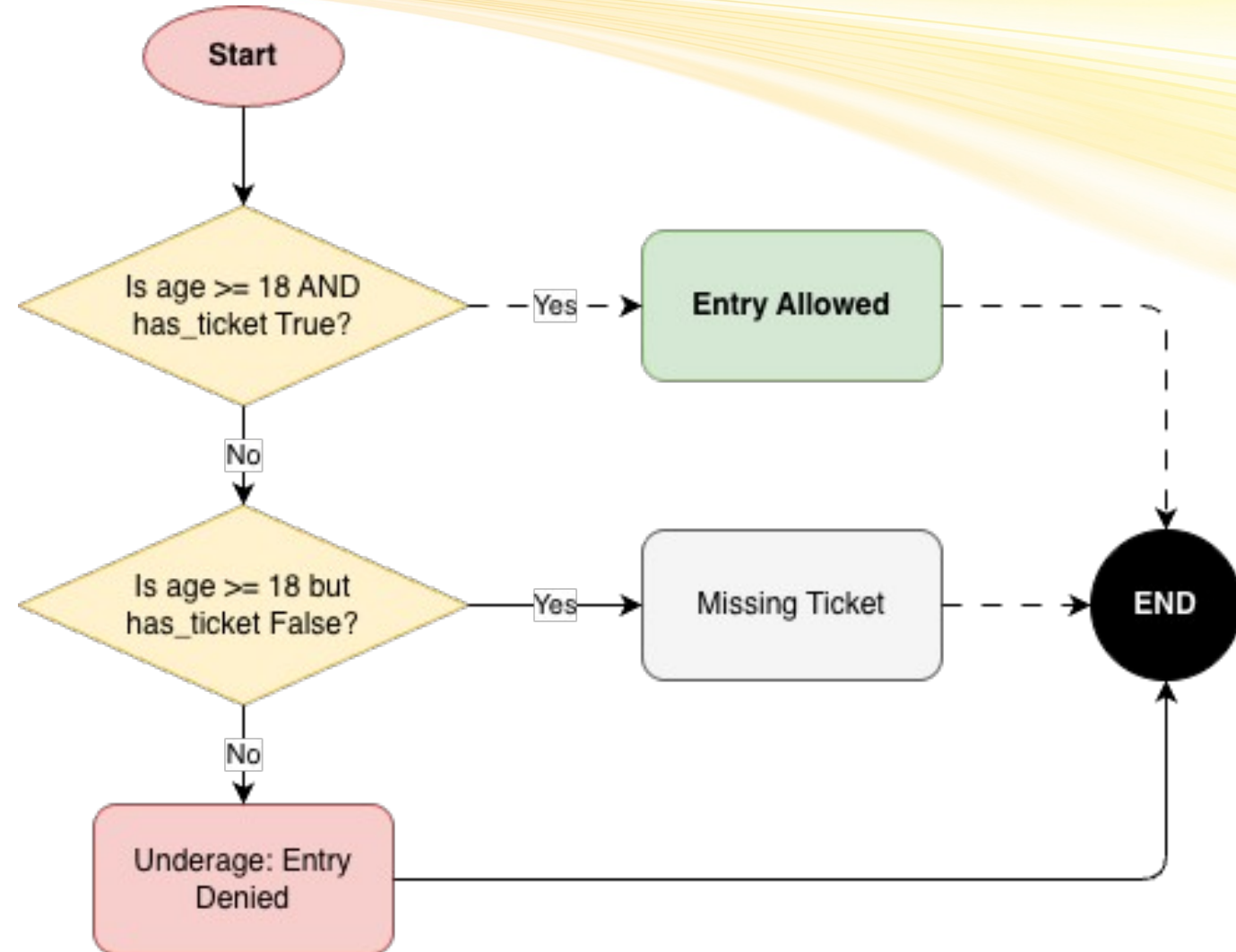
Discount Calculation

```
IF order_total >= 100 THEN
    discount = 20
ELSE IF order_total >= 50 THEN
    discount = 10
ELSE IF order_total >= 0 THEN
    discount = 0
ELSE
    DISPLAY "Error: Invalid Negative Total"
ENDIF
```



Simple Eligibility Checker

```
IF user_age >= 18 AND has_ticket == True THEN  
    DISPLAY "Entry Allowed"  
ELSE IF user_age >= 18 AND has_ticket == False  
THEN  
    DISPLAY "Missing Ticket"  
ELSE  
    DISPLAY "Underage: Entry Denied"  
ENDIF
```



Translating Logic into Code

The Core Strategy

- **Direct Mapping:** Convert flowchart shapes or pseudocode branches straight into if, else if, and else statements
- **Maintain Alignment:** Keep the code structure perfectly mirrored to your original design layout
- **Avoid Improvisation:** Write your script exactly as planned before optimising the logic

Testing and Refining Your Script

Now, you need to convert your flowchart or pseudocode into Power Fx Formula:

Practical Code Review

- **Targeted Testing:** Run test variables through every condition path
- **Trace Outcomes:** Confirm that actual script execution matches intended flowchart paths
- **Bug Squashing:** Refine syntax gaps and eliminate any logic holes

The Ultimate Goal

- **Working Code:** Scripts must execute flawlessly without error
- **Accurate Logic:** Your program fully handles every predefined business condition

Application Build Task



ACTIVITY



Enhance your Task Entry App By:

- Adding validation logic, e.g. required fields
- Displaying conditional messages

Tasks:

- Create simple decision outputs based on input



ACTIVITY



The Next Activity will begin soon



Request Approval App



ACTIVITY



Objective: Build an app that applies approval rules based on an amount

Tasks / Requirements:

- Define rules based on amount thresholds
- Implement the conditional logic using If()
- Display a different outcome message based on the input

Key Point: Refactor multiple nested conditions into a clearer structured formula

Extension Activity: Add a second condition based on requester role



ACTIVITY



The Next Activity will begin soon



Debugging and Refinement Activity



ACTIVITY



Common Issues Provided:

- Incorrect condition ordering
- Missing cases
- Overly complex nested logic

Tasks:

- Identify and annotate issues
- Simplify and restructure conditionals where appropriate
- Correct and re-run the script

Expected Outcome: Improved scripts demonstrating clear, efficient and correct conditional logic



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion



Complex vs. Simplified

- **The Anti-Pattern (Complex):** Deeply nested, multi-layered If() statements. They are highly prone to syntax errors and difficult to read
- **The Best Practice (Simplified):** A single, flattened Power Fx If() block using clean line breaks and sequential arguments

// COMPLEX (Deeply Nested Anti-Pattern)

```
If(Value(Amount.Text) > 1000, "High",  
If(Value(Amount.Text) > 500,  
If(Value(Amount.Text) > 100, "Medium", "Low"),  
"Low"))
```

// SIMPLIFIED (Flattened Best Practice)

```
If(  
Value(Amount.Text) > 1000, "High",  
Value(Amount.Text) > 100, "Medium",  
"Low"  
)
```

The Cost of Complexity



USABILITY



MAINTAINABILITY



DEBUGGING SPEED

Stretch and Challenge

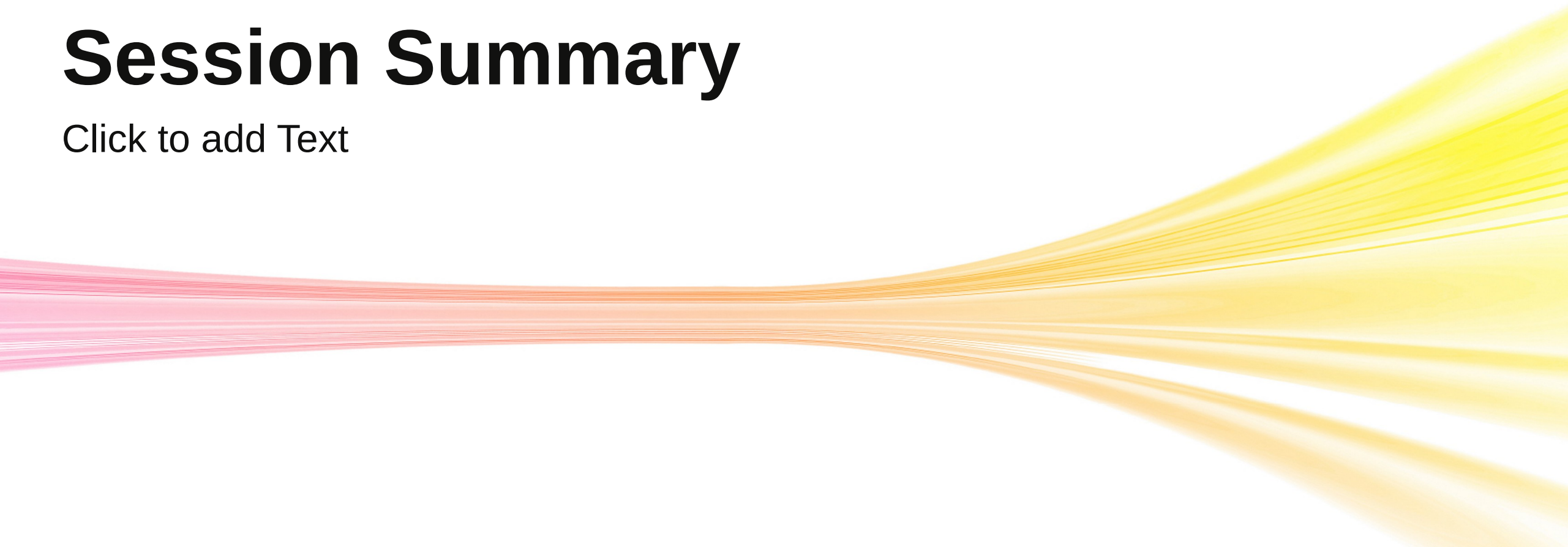
The Mission

Improve your solutions by doing the following:

- Refactor multiple conditions into clearer structured formulas

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Conditional Logic and Decision Structures

Lesson Objectives

Apply

- Apply conditional logic within Power Apps formulas

Analyse

- Analyse decision logic using flowcharts

Evaluate

- Evaluate logic for clarity and usability

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

